



OWNER & MAINTAINER HANDBOOK · EDITION 01

Specialty Match

Technical handover

The technology, the research rules, and the practical knowledge you need to return confidently after a year.

81

self-assessment items

58

specialty narratives

3

languages & audiences

The project in one sentence

A multilingual medical-exploration application built with React and TypeScript, with a versioned matching engine, Supabase research storage and staff authentication, an administrative portal, and shared web/mobile code.

Read this first

This is a **repository snapshot dated 25 September 2026**, based on Git revision 5be6d2f and the accompanying source, migrations and test reports. A file existing in the repository does not prove that it has been deployed.

The live database, provider accounts and app stores were not contacted to verify release status while preparing this PDF. The guide contains **no passwords, API-key values, access tokens, patient information or research responses**.

Most important rule: verify and update the database before publishing a frontend that needs new RPCs. Back up first. Never reset production to make local tests pass.

Project names used in files: Specialty Match / Q Project / Q-pro. The npm package still has its original starter name; that is not the product's research or release version.

Your first hour back

1. **Recover ownership.** Locate GitHub, Supabase, hosting/DNS, backup, SMTP and signing-key access through your password manager. Check who can recover each account and its MFA.
2. **Identify what is actually live.** Compare the deployed Git SHA, hosting release and database migration history. Do not assume `main`, this PDF or the latest local folder matches production.
3. **Protect the current data.** Confirm the latest encrypted backup and successful restore drill. Do not upgrade, migrate, clean up or delete data before this check.
4. **Reproduce the recorded environment.** Start from the lockfile and Node 24 baseline, in a branch. Use synthetic staging data, not the real research database.
5. **Run the checks.** Frontend tests are not substitutes for real Supabase database/security tests. Check access, submission receipts, map visibility and exports.
6. **Review the changes since this date.** Runtime support, security advisories, provider prices, consent rules and native-store requirements will have moved. Plan upgrades separately from recovery.

Do not “fix everything” with a bulk dependency update. First reproduce the baseline; then upgrade one component at a time, with tests and a rollback plan.

Where to look in this guide

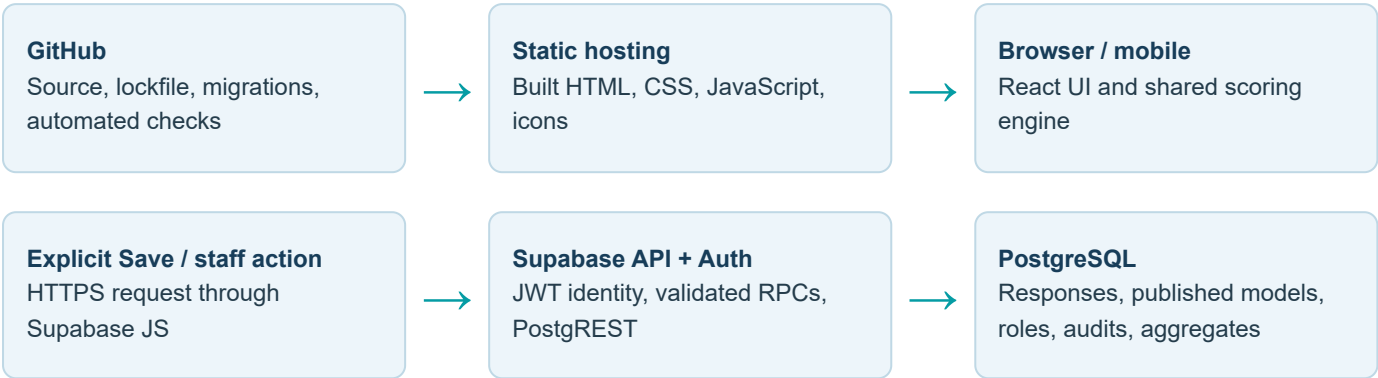
- | | |
|-------------------------------------|-----------------------------------|
| 03 Architecture and data flow | 11 Evaluation and private exports |
| 04 Technology and version inventory | 12 Deployment and environments |
| 05 Repository and local setup | 13 PWA and native applications |
| 06 Product rules and content | 14 Verification and release gates |
| 07 Matching engine and provenance | 15 Backups, maintenance and costs |
| 08 Database and API structure | 16 Incidents and owner checklist |
| 09 Access, privacy and secrets | 17 Reference map and glossary |
| 10 Dashboard and map operation | |

Source-of-truth order

For a live incident: actual deployed revision and database state → matching source/migrations → latest dated test and operations reports → older documents. This PDF is a map, not a replacement for those records.

Do not treat an old chat, saved login code or copied project URL as authoritative configuration. Backend URLs can change, and native bundles may still point to an earlier project.

One application, clear boundaries



What runs where?

Layer	Responsibility
Public client	Questionnaire state, translations, specialty explorer, compatibility scoring and results. No privileged server key.
Research client	Authorized staff controls and paginated data. Core cohort statistics are database-computed, not a whole-database browser calculation.
Database	Durable writes, idempotency, validation, access control, catalog publication, map release, bounded analysis records.
Operator workstation	Explicitly authorized, bounded advanced analysis/export with the same TypeScript scoring code; private output files.
Native shell	Capacitor wraps the built web app and connects secure storage/lifecycle. It does not introduce a second matching algorithm.

A submission’s lifecycle

Select an audience → answer in memory → obtain the ranking with a published catalog → choose whether to save with consent → freeze the submission UUID, answers, catalog and scoring context → send or queue it → call it saved only when the server returns that same UUID.

Retrying the same accepted payload returns the original receipt. Reusing its ID with different content must fail, not silently update the research row.

Not part of this architecture: no required AI API, trained-model service, dedicated Express/Next.js server, Redis, Kubernetes or public response-file bucket. A static frontend host does not provide database backups.

Primary code: `src/App.tsx`, `src/lib/supabase.ts`, `src/lib/scoring.ts`, `supabase/migrations/`.

The stack you are maintaining

Versions below are the repository's locked baseline, not a claim that they are the latest or still supported when you return. Several manifest entries use ranges; `package-lock.json` fixes the resolved dependency tree.

Technology	Baseline	Why it is here
React + React DOM	18.3.1	Components, state and the web interface.
TypeScript	5.6.3	Typed application, scoring, API contracts and tests.
Vite	5.4.8	Development server and static production builds.
Tailwind CSS	3.4.17	Utility styling; PostCSS 8.4.47 and Autoprefixer 10.4.20 process CSS.
Lucide React	0.446.0	Interface icons; the product logo is a separate vector asset.
Supabase JS	2.57.4	Auth sessions, database reads and RPC calls.
PostgreSQL / PostgREST	Local PG 17	Relational data and SQL-backed HTTPS API. Verify the live database version independently.
Capacitor core / CLI / platforms	8.5.2	Android/iOS wrappers; App plugin 8.0.1 adds lifecycle integration.
Capacitor secure storage	8.0.0	<code>@aparajita/capacitor-secure-storage</code> for native protected storage.
Playwright	1.63.0	Browser workflows, responsive checks and PWA tests.
ESLint / esbuild	9.12.0 / 0.21.5	Linting and bundling isolated tests/operator worker code.
Node.js + npm	Node 24 CI	Build/test/operator tools. Node is not the deployed static website server.
Supabase CLI + Docker	CLI 2.115.0 in CI	Disposable local backend, migrations and real database tests.
pgTAP / pg_cron	DB extensions	Database assertions and monthly map publication.

Map: custom SVG with bundled Natural Earth geometry; no Leaflet, tile-server API key, GPS request or commercial map SDK in this checkout.

Other formats/tools: SQL/PLpgSQL migrations, Git/GitHub Actions YAML, JSON/CSV research exports, HTML/Markdown documentation, SVG/PNG/ICO assets. Python is used by an existing Word-document generator, not the production scoring engine.

Find things quickly

Path	What to change or inspect there
src/main.tsx · src/App.tsx	Bootstrapping, providers, audience and page/quiz orchestration.
src/components/	Questionnaire, results, explorer, dashboard, map and editors.
src/data/	81 items, traits, values, specialties, translations, narratives, geography.
src/lib/scoring.ts	Shared deterministic matching calculations.
src/lib/researchDashboard.ts researchEvaluation.ts	Eligibility, canonical analysis, export serializers and extended evaluation.
src/lib/supabase.ts database.types.ts	Client configuration, validated submissions, queue transport and database types.
supabase/migrations/ · tests/	Ordered database history and SQL security/regression tests.
public/ · scripts/	Static assets/service worker; generators, verification and operator tools.
android/ · ios/	Actual native projects, platform manifests and signing-related configuration.
docs/ · .github/workflows/	Handover/operations evidence; separate frontend and database CI.

Recreate a safe development session

```
git status --short
node --version
npm --version
npm ci
# Create .env from .env.example using a SYNTHETIC development project.
npm run dev
```

Use Node 24 to reproduce the recorded baseline. On Windows PowerShell, `npm.cmd` can be used if script execution policy blocks `npm`. Open the URL Vite prints; the default base is `/Q-pro/`.

`.env` is ignored by Git. Only a public backend URL and publishable key belong in `VITE_*` variables. Without a separate development backend, leave remote write configuration unset and use mocked tests instead of production.

Preserve `package-lock.json` and existing user changes. Do not rename historical migrations—including the old `.sql.sql` filenames—or replay migrations on a restored schema without reconciling its history.

Behavior to preserve

Three audiences

Before medicine: curious explorers.

During medicine: medical students.

After medicine: specialists.

Audience selection is a questionnaire category, never an administrative permission.

Three languages

English, Romanian and French use repository translation dictionaries. Change every relevant language together; retain stable question IDs and canonical specialty names.

Questionnaire and results

- The main questionnaire contains **81 ratings on the 1–10 scale**; career values and five dimension priorities are separate inputs. Mobile controls must not change the underlying numeric scale.
- Students and explorers may add an **optional** written perspective on medicine. Saving remains a choice. A student's optional study year is **1–6**; explorers have no study year.
- Specialists can answer the main questionnaire or skip directly to the qualitative interview. Skipping is explicit and produces no invented quantitative ranking.
- The specialist interview asks about the specialty now, changes over time, its most important quality, choosing it again, and a question a student should ask. A “no” rechoice requires an explanation.
- Results show the top 10 specialties with narratives and bibliography where supplied. Do not reinstate removed calculated-analysis or empty-reference sections by accident.
- Back navigation and scroll-to-top are shared behaviors. Unfinished answers remain in memory across in-app navigation but are not restored after reload or closing.

Editorial ownership

The Romanian source is at `docs/sources/specialist-specialty-narratives-ro.txt`. It has 58 numbered sections but **57 unique specialties**: two Pulmonology sections merge; Pathology is absent. Do not label every catalog narrative specialist-authored. `src/data/specialtyNarratives.ts` and its JSON parts preserve provenance and translations.

The staff editor can change versioned descriptions, clinical summaries and target profiles without editing the code. Only authorized publication makes a draft active.

Branding uses `BrandLogo.tsx`, `public/branding/compass.svg` and generated web/native icons. `npm run generate:icons` regenerates assets; `test:icons` checks them. The supplied raster artwork is retained, not the only UI logo format.

Check `src/config/features.ts`: public research CTA, specialty comparison, methodology and opposite-fit controls are currently off. A component existing in the code does not mean its public entry control is enabled. Navigation uses `appNavigation.ts`, not React Router.

Understand what the score means

This is a **deterministic compatibility engine**, not an AI diagnosis, a probability of career success or an objectively tested measure of competence. Changing a specialty profile changes a model configuration, not proof of validity.

- 1. **Normalize ratings:** $((\text{answer} - 1) / 9) \times 100$. Trait mappings define weights and forward/reverse alignment.
- 2. **Build measured traits:** weighted combinations of mapped answers. Selected career values do not inflate an already measured trait's score; they adjust matching importance.
- 3. **Handle value-only evidence:** a selected value can introduce a trait absent from item measurement. The current selected value-only level is 90. Unmeasured traits are not silently replaced with 50.
- 4. **Compare with each specialty:** combine trait similarity with profile importance, dimension-priority multiplier and career-value multiplier. Divide by contributing weights. Missing dimensions can be unavailable rather than a false zero-fit.
- 5. **Rank at full precision:** display rounding must not create research ties. A preferred or actual specialty is not an input to the score.

Identifier / concept	Recorded baseline
Questionnaire / values / specialties	q81-v1 / career-values-v1 / medical-specialties-v1
Engine / trait mapping	scoring-engine-v2 / question-traits-q81-v1
Participant scoring / calibration	client-scoring-v2 / calibration-v2-qualitative
Canonical research analysis	dashboard-canonical-default-v3: exact full-precision ties.
SQL item analysis	research-sql-q81-v1: original answer statistics, not a second matching engine.
Runtime catalog	Immutable published UUID + revision; checksum identifies the profile/mapping configuration.

Structural limits remain: manual_orientation and prevention_orientation are not uniformly measured across the cohort. Top-k fit alone cannot justify changing those targets or claiming validation.

New scoring_context records actual five-slider settings and engine/mapping/model identifiers at Save. Historical missing settings stay null. Canonical evaluation explicitly uses default priorities; it does not reconstruct what an older participant saw. Client-reported context is provenance, not server attestation.

Three schemas, distinct responsibilities

Main objects	Purpose and relationships
<code>public.student_responses</code>	Students and explorers share this table; <code>participant_role</code> distinguishes them. Ratings, values, optional reflection/year/geography, stored scores and provenance.
<code>public.specialist_responses</code>	Actual specialty, optional quantitative participation, qualitative interview, original metadata and scoring context.
<code>auth.users</code> <code>private.researchers</code>	Managed staff identity plus enabled application allowlist/role. They are not public participant profiles.
<code>private.trait_catalog</code> <code>specialty_catalog_versions</code> <code>specialty_catalog_entries</code> <code>specialty_catalog_audit</code>	Versioned editorial models and audit trail. A version has entries; responses retain the published version UUID/revision they used.
<code>private.participation_map_months</code> <code>participation_map_cells</code>	Sealed monthly public releases and suppressed/rounded cells; no live public query of raw locations.
<code>private.public_features</code> <code>public_features_audit</code>	Central public-map switch and actor/time/previous/new-value history.
<code>private.research_cohorts</code> <code>research_analysis_runs</code> <code>research_request_windows</code>	Owner-specific live filters, frozen aggregate results/member digests, per-account throttle counters.
<code>private.portal_test_dataset</code>	Removable synthetic generation recipe. Not 15 inserted research responses and not real consent.

Important API boundaries

`get_active_specialty_catalog` reads published content; `current_user_portal_profile` checks staff access. New client submissions use **participant v7** and **specialist v6**. They retain collection schema **5** and **3** respectively: an RPC version and a row schema version are different things.

The public and private map endpoints are now separate. Research functions provide bounded pages, aggregates, saved filters and recorded runs. Keep old submission RPCs for compatible historical/queued clients unless a reviewed deprecation plan says otherwise.

An exposed schema is not unrestricted access. Raw research tables rely on explicit grants and RLS; private tables are accessed through narrow authorized RPCs. Never make them public to work around a permission error.

Primary references: `src/lib/database.types.ts`, `docs/BACKEND_AUDIT.md`, all 18 migration files in this snapshot. The generated/manual client types must remain aligned with SQL.

Identity is not authorization

Actor	Allowed scope
Public participant	Published catalog, validated consented submission, approved public aggregates only when enabled. No raw research reads.
Ordinary signed-in user	Authentication alone grants no research permission. Being a specialist respondent grants no staff access.
Researcher	Authorized research reads, analyses/exports and private map; owner-specific saved filters/runs.
Doctor	Research access plus catalog draft editing and public-map settings; synthetic recipe controls.
Professor	Doctor capabilities plus governed catalog publication and restoration.

Rules that must survive future refactoring

- Authorize on the server using enabled `private.researchers` membership. Do not trust audience selection or user-editable JWT metadata.
- Use explicit grants, RLS and a restricted function search path. Privileged SQL functions need role checks and revoked default public execution.
- Never put secret/service-role keys, database passwords or operator tokens in `VITE_*`, the mobile bundle, Git, screenshots or this handbook.
- On expiry or revocation, clear private UI state and deny subsequent API calls. Hiding controls is not access control; previously downloaded files cannot be remotely erased.
- Do not log answers, narratives or tokens. Free text can identify someone even when names/emails are not requested. “No name field” is not a blanket anonymity claim.

Local storage is deliberately split

Unsubmitted questionnaire drafts: memory only; no save/resume panel. **Explicitly consented pending submissions:** a separate frozen retry queue persists until sent, rejected, expired or removed. Browser storage is not encrypted; native storage uses the secure-storage adapter. A queued contribution is not yet saved on the server.

The current queue is bounded (10 entries and approximately one million serialized characters); automatic retry ends after seven days. Do not clear all browser data casually if an unsent consented contribution may still be present.

Outstanding: approve consent/provenance disclosures, retention/deletion and research governance; implement approved anonymous-submission anti-abuse before broad recruitment. The staff analytics rate limit is not CAPTCHA or an Internet firewall.

What each workspace is for

Area	Purpose / caution
Specialists	Paginated qualitative/quantitative records, eligibility and response details. Skipped questionnaires remain useful interviews, not quantitative training examples.
Students & explorers	Separate audience filters, optional study year/reflection, preferences and stored browser rankings. Preferences are not professional ground truth.
Research analyses	Server-side recruitment counts, monthly/category/language/country/version groups, item distributions, Pearson correlation, saved cohorts and recorded runs.
Participation map	Authorized private aggregate map with respondent, country, language, month range and data-version filters.
How the algorithm works	Visual explanation of mappings, priorities, scoring and limits.
Versioned configuration	Edit descriptions, summaries and target-profile drafts; publish/restore through separate role-governed actions.
Settings · Public features	Audited public-map visibility switch, available only to authorized editors.

The map may correctly be empty

The public switch defaults to disabled and fails closed when it cannot be checked. Disabled means no public links, previews, geography widgets or public data endpoint access. Authorized private research access remains separate.

When enabled, the map shows **published submissions, not live totals or unique people**. Only eligible closed-month cells with at least 10 submissions are released, rounded down to multiples of 5. Missing geography, the current month, suppressed cells and skipped specialist questionnaires do not become public counts.

The monthly release is sealed and cannot be casually rewritten. `pg_cron` schedules publication at 01:15 on the first day of each month; check its history and UTC month boundaries. A newly submitted record need not appear immediately.

Testing without polluting research

Authorized editors can create/remove a recipe generating 5 specialists, 5 students and 5 explorers. Test mode stays separate, marks exports synthetic, and blocks live server analyses/settings changes. Remove the exact recipe through the UI; never delete real response tables to “clean up test data.”

A visibility check is refreshed on lifecycle events and every 30 seconds. Offline/failed checks hide the public map. Approved data previously seen by someone cannot be clawed back.

Reproducible does not mean validated

Live cohort

A saved named filter. New submissions can change its membership and counts.

Frozen result

A recorded analysis with filters, parameters, digest, model versions, time and derived results. Not a full raw-data backup.

Always report the denominator: global submissions, filtered submissions, eligible questionnaires and exclusions are different totals. Missing optional metadata does not automatically exclude a valid questionnaire. Cohort comparisons may overlap and are not automatically independent samples.

What the analyses calculate

- **Items:** original 1–10 bins, n, missingness, mean, median, sample SD, floor/ceiling counts. Pearson correlations use eligible complete questionnaires; constant items give no coefficient.
- **Specialist evaluation:** Top-1/3/5/10 recall, median rank and mean reciprocal rank. Inclusive ties use best rank, conservative ties worst rank. No rounded-score ties.
- **Groupings:** submission-weighted and equally weighted observed-specialty summaries; language/metadata strata and trait profiles/distributions. Review flags never automatically remove people or questions.
- **Not claimed:** population confidence intervals without a sampling basis, validated latent traits, factor analysis, language invariance or reliable unique-person counts.

Choose the right export path

Path	Bounds and interpretation
Browser export	Up to 1,000 filtered submissions; raw-wide, long, analytical CSV or JSON; metadata and SHA-256. Created-at cutoff/keyset pagination is not an atomic database snapshot.
Server analysis	Up to 5,000 submissions, 20 retained results per owner, 10-second SQL timeout; 50 named cohorts. 30 successful expensive operations per five minutes per account.
Operator worker	5,000 rows / 32 MiB input / 128 MiB output; progress, cancellation and three-minute default overall deadline. Before/after cohort checksum comparison; private files outside Git.

The operator uses an authorized staff access token, not a service key, and evaluates the explicitly frozen **repository-source catalog** unless a separate candidate is supplied. It does not silently use the live published model. Candidate comparison fits no parameters and promotes nothing.

Use `npm run research:export -- --help` and `docs/RESEARCH_OPERATOR.md`. Distribute CSVs only with their manifest/model provenance. Digests are integrity labels, not encryption or anonymization. Training a future model requires fitting inside training splits and independent evaluation.

Database first, then the frontend

Build target	Command / output	Asset base
GitHub Pages	npm run build → dist/	/Q-pro/
Root-hosted website	npm run build:hosted → dist/	/
Native bundle	npm run build:mobile → dist-mobile/	./

GitHub remains source control and CI. A Pages workflow is configured on `main`; Cloudflare Pages is the documented recommended alternative, **not a confirmed active account/deployment**. Hosting changes do not migrate or fix the database.

Environment variables: names, never secrets

Variable	Where / meaning
VITE_SUPABASE_URL	Public frontend build-time project URL.
VITE_SUPABASE_PUBLISHABLE_KEY	Public client key; authorization still belongs to SQL/Auth.
VITE_SUPABASE_ANON_KEY	Legacy fallback only, not a secret key.
RESEARCH_SUPABASE_URL RESEARCH_SUPABASE_PUBLISHABLE_KEY RESEARCH_ACCESS_TOKEN	Operator-only environment. The access token is secret, short-lived and never a frontend variable.

Controlled release sequence

1. Confirm project identity, backup/restore readiness, role owners and an isolated synthetic staging environment.
2. Run frontend and real database tests; review applied versus pending migrations. Approve the production change.
3. Apply reviewed database changes; verify new RPCs, grants, denial cases and durable receipts.
4. Build with the correct host base and public backend settings. Test a preview, then promote it.
5. Record Git SHA, migration list, catalog revision, deployment ID, test evidence and rollback decision.

CI coordination gap: database tests and Pages deployment are separate workflows. Pages does not wait for the database workflow or apply migrations. A push to `main` can publish a frontend too early; use a branch and explicit release gating.

Changing build-time environment values requires a rebuild. New domains need reviewed Auth origins/redirects. Preview writes must use a synthetic backend. Do not copy a production service key into CI or mobile assets.

Shared code, separate releases

Installable web application

The manifest and generated icons provide an installable identity. `scripts/pwaBuildPlugin.ts` inserts a content-versioned allowlist into `public/sw.js`. Production web builds register it; development and Capacitor builds do not.

Only exact public application assets enter this cache. API, settings, map, staff responses, auth and exports remain network-only. Offline map access is not an entitlement. New workers wait rather than force-reload an unfinished memory-only questionnaire.

Do not confuse public catalog caching with answer persistence. The current client can reuse a validated published catalog for up to 30 days. A failed refresh can still leave a usable cached catalog for a new assessment. Saving needs server acceptance of the referenced published version. This is not permission to restore old answers with a changed model.

Native projects in the repository

Configuration	Recorded baseline
Identity	<code>ro.qpro.specialtymatch</code> · Specialty Match.
Web bundle	<code>capacitor.config.ts</code> points to <code>dist-mobile</code> ; do not replace it with an arbitrary <code>remote.server.url</code> .
Android	<code>android/</code> ; recorded min SDK 24 and target/compile SDK 36; documented JDK 21 toolchain. Recheck future requirements.
iOS	<code>ios/</code> ; minimum deployment baseline 15; macOS/Xcode and signing are required for actual archive/device release.
Protected storage	Native secure-storage adapter for sessions and consented queue; browser <code>localStorage</code> is not equivalent protection.

```
npm run build:mobile
npm run mobile:sync
npm run mobile:android # opens Android Studio
npm run mobile:ios     # requires macOS/Xcode
```

Build status is layered: web assets built ≠ native compile passed ≠ device tests passed ≠ signed artifact created ≠ store published. The recorded handover does not prove signed Android/iOS releases.

App/universal links and OAuth/password-recovery callbacks are not configured. Adding them needs an owned domain, verified platform associations and safe allowlisted handlers. Review keyboard, back navigation, safe areas, lifecycle, secure storage, offline behavior and privacy declarations on real devices.

What to rerun before trusting a release

```
npm run typecheck
npm run lint
npm run test:dashboard
npm run test:map
npm run test:portal-data
npm run test:mobile
npm run test:icons
npm run test:specialty-content
npm run test:research-operator
npm run test:browser
npm run test:pwa
npm run build
npm run build:mobile
```

Read each script before running it against any configured backend. The named browser/operator regression suites use synthetic fixtures; similarly named runtime scripts may contact a configured project. A production smoke test needs deliberate scope and authorization.

Recorded evidence—not new verification for this PDF

The 25 September implementation report records passing frontend, browser, PWA and web/mobile-asset builds. Local PostgreSQL/WASM checks passed 82 map assertions, 50 synthetic-data assertions, 72 research checks, 29 provenance checks and 18 throttle checks, plus 10 benchmark checks.

The original `001_research_security.sql` could not run in that limited harness because full pgTAP helpers were missing. Docker was unavailable during the recorded work. Real Supabase HTTP/Auth behavior, response headers, independent-session races and advisors therefore remain release checks.

Database verification

`.github/workflows/database-tests.yml` starts a disposable Supabase stack and runs pgTAP plus database lint. Locally, start Docker and use the compatible CLI; inspect `--help` before discovering `start/test/lint/advisor` commands. Never run a reset against the linked production project.

Priority backlog

- Real integration tests and backed-up migration rollout; privacy/consent sign-off and anonymous anti-abuse.
- Hosting/account/domain setup and actual release verification; monitoring ownership and tested recovery.
- Native compile/device/signing/store checks; safe deep links if required.
- Reduce the large public bundle only after measurement: recorded ~897 KB minified / 265 KB gzip. Advanced analysis is operator-run, not a durable dashboard job queue.

Synthetic benchmarks show feasibility, not production capacity: ~1,000 rows were tested locally. See the dated report for timings, bounds and untested areas.

Protect the data before optimizing costs

A backup is only useful if it restores

Assign an owner. Keep encrypted off-site backups outside Git and public storage. Preserve schema, data, relevant roles/configuration, migration history, Auth/SMTP settings, recovery material and storage objects if any. A CSV research export is not a database backup.

Rehearse restoration into an isolated compatible environment with outgoing email/hooks disabled. Check integrity, row counts, roles/RLS, published versions, consent, idempotency and the deletion ledger. Do not replay every migration over an already current restored schema.

Documented pilot targets—not guarantees—are at most 24 hours of recoverable data loss during recruitment and recovery within one business day. The proposed rotation is seven daily/four weekly encrypted copies, with monthly restore drills, subject to approved retention obligations.

Maintenance rhythm

When	Review
During recruitment	Backend availability, save failures, queue reports, fresh backups and anonymous abuse.
Weekly	Database/egress growth, quota forecasts, slow queries, account changes, security/dependency notices and invoices.
Monthly	Restore drill, staff access review, retention/deletion, map publication history and deployment inventory.
Before every release	Backup, tests, migration order, preview checks, consent/content changes and rollback compatibility.

Cost snapshot: verify again when you return

Official pages checked 25 September 2026: Supabase Free lists a 500 MB database, 1 GB file storage, 5 GB egress plus 5 GB cached egress, 50,000 Auth MAU, inactivity pausing after one week, and no automatic backups. Pro starts at \$25/month; extras and taxes are separate. Public visitors are not automatically Auth MAU. [\[S1\]](#)

Cloudflare Pages Free documents 500 builds/month, 20,000 files and 25 MiB per file. Static hosting is not unlimited backend capacity. Spending controls do not cover every explicitly provisioned Supabase add-on. [\[S2\]](#) [\[S3\]](#)

Planning warnings: review growth at 300 MB database / 3 GB forecast monthly egress, before approaching free limits. These are internal warning thresholds, not provider promises. Domains, SMTP, app-store accounts and operational time need separate ownership/budget.

When something stops working

Symptom	First safe checks—not a blind fix
Blank site / missing assets	Inspect deployment logs, loaded asset paths and correct base (/Q-pro/, /, ./). Confirm build-time backend settings without exposing keys.
Login works, portal denied	Check enabled staff allowlist/role and expiry. Ordinary authentication is insufficient. Do not weaken RLS.
Save fails / RPC missing	Compare frontend RPC versions with applied migrations, project health and quota. Preserve the immutable queued payload; never show success without a receipt.
Public map hidden / empty	Check visibility, fresh online authorization, sealed months, small-cell suppression, geography and cron history. Do not expose raw rows to fill the map.
Results look different	Compare engine, mappings, catalog UUID/revision, priorities and analysis mode. Default-priority research output may differ from personalized results.
Analysis refused or slow	Check role, 429 retry delay, cohort/run caps and timeout. Narrow filters; use the bounded operator workflow when appropriate.
Old UI after release	Check deployed SHA and service-worker lifecycle. Ask users to finish or knowingly discard open answers before closing all app windows and reopening online.
Suspected leaked access	Use the incident process: revoke affected access/sessions, rotate exposed privileged secrets, preserve restricted evidence and verify direct API denial.

Record privately before you leave

Use a password manager or approved private operations register—not this PDF—for these locations and recovery details:

- Repository owner, production URL, active hosting project and deployed SHA.
- Supabase project reference/region, actual migration list, billing owner and staff approvers.
- DNS/domain and SMTP owners; MFA/recovery contacts; authorized token/secret custody.
- Backup destination, encryption-key custodian, last successful restore and deletion ledger.
- Android signing-key custody; Apple team/certificates; store listings and latest signed build IDs.
- Research owner, consent/retention approval, incident contact and budget approval limit.

Frontend rollback does not roll back the database or installed native apps. Prefer reviewed forward database fixes that preserve raw responses and audit history. Restoring an old backup requires reconciling later accepted submissions and deletions.

Where the detailed answers live

Repository document	Use it for
<code>docs/BACKEND_AUDIT.md</code>	Current RPCs, schema additions, eligibility, security, rate limits and SQL benchmarks.
<code>docs/ANALYTICS_METHODS.md</code>	Denominators, ties, quality flags, provenance and scientific limits.
<code>docs/RESEARCH_OPERATOR.md</code>	Private bounded exports, token handling, manifests and candidate comparisons.
<code>docs/ARCHITECTURE_AND_BUDGET.md</code>	Two hosting options, workload assumptions, provider limits and upgrade triggers.
<code>docs/OPERATIONS.md</code>	Deployment, monitoring, encrypted backups, restore and rollback.
<code>docs/MOBILE.md</code>	Native/PWA toolchains, signing, privacy and release checklist; verify cache behavior against current code.
<code>docs/IMPLEMENTATION_TEST_REPORT_2026-09-25.md</code>	Dated test evidence, remaining work and manual release gates.

Known documentation drift: older `SUPABASE.md`, database Word/Markdown guides and `PARTICIPATION_MAP.md` describe earlier RPCs or map permissions. Current code uses participant v7 / specialist v6 and a separate private-map RPC. An older claim that every new quiz needs an online catalog is too absolute because validated public catalog caching now exists.

Official references to revisit

[S1 · Supabase pricing](#) · [S2 · Cloudflare Pages limits](#) · [S3 · Supabase cost controls](#)
[Supabase backups](#) · [Backup/restore procedure](#) · [Supabase changelog](#)
[Node release lifecycle](#) · [Vite](#) · [React](#) · [Capacitor](#) · [Playwright](#)
[Apple review guidelines](#) · [Google Play target API policy](#)

Small glossary

RPC: a database function exposed through an API. **RLS:** row-level security. **JWT:** signed identity/session token, not automatic permission. **Migration:** ordered database change. **PWA:** installable web app. **Catalog:** versioned specialty content and target profiles. **Cohort:** selected submissions. **Idempotency:** repeat the same write safely without duplicating it. **Checksum:** integrity/version identifier, not encryption. **CI:** automated checks/builds. **RPO/RTO:** acceptable data-loss window / recovery-time objective.

Editable source: this handbook's matching HTML file in `docs/`. Regenerate with `node scripts/generate-handover-pdf.mjs` after reviewing the dated content. Update the edition/date and reverify facts before treating a regenerated PDF as a new handover.